

Chapter 8. MLflow for GenAI

MLflow for GenAI can be thought of as a *distinct layer* on top of the MLflow concepts we've seen so far. While familiar building blocks still apply, such as experiment tracking for logging runs and registering models, the application of these features shifts significantly in the context of generative AI.

In GenAI workflows, we still log and register models, but we rely on the PyFunc flavor with a different abstraction. Instead of extending *PythonModel*, GenAI applications typically use classes such as *ResponsesAgent*, which are designed to represent agents rather than traditional predictive models.

Evaluation is also fundamentally different. Rather than focusing on standard ML metrics, GenAI evaluation centers on behavioral qualities and agent decisions. To support this, MLflow introduces *traces* to provide fine-grained observability into everything that happens inside an agent, including tool calls, intermediate steps, and execution flow in real time.

In the following sections, we'll explore MLflow's GenAI flavors and the fundamentals of MLflow tracing. From there, we'll define a custom agent with tracing enabled and progressively build toward evaluation, logging, and registering the agent.

MLflow Flavors

In [Chapter 3](#), we explored logging and registering ML models using MLflow, and we discussed different built-in flavors like scikit-learn (sklearn), LightGBM, and others. In the same way, MLflow provides specialized flavors for agentic frameworks, including DSPy, LangChain, and LlamaIndex. These libraries simplify the orchestration of AI agents by providing high-level abstractions for components like tool use, memory, and LLM calling.

For even greater control, you can define your own custom orchestration logic and utilize `mlflow.pyfunc.ResponsesAgent`, which follows OpenAI Responses API. This approach gives you more flexibility than us-

ing built-in flavors, and it can be great for fast prototyping. [Figure 8-1](#) summarizes these various flavors and illustrates the expected payload format when these models are deployed to a Mosaic AI Model Serving endpoint.

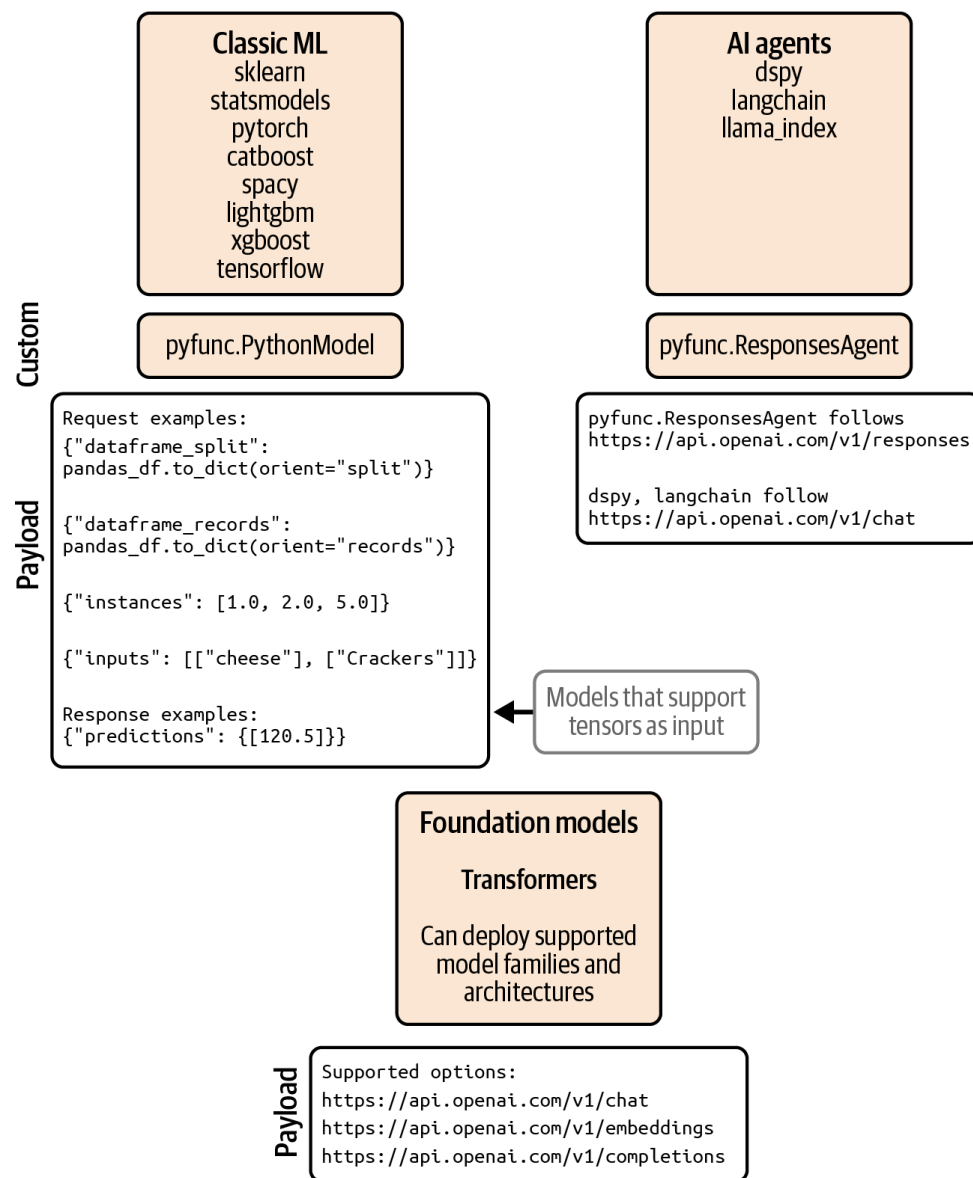


Figure 8-1. Model flavors and payload

In this chapter, we'll focus on building a custom agent. However, before we jump in, let's take a step back and look at the theory behind MLflow tracing, since we'll rely on the `@mlflow.trace` decorator when building the agent.

MLflow Tracing

MLflow tracing gives you visibility into what your LLM application is actually doing under the hood. It's an OpenTelemetry-compatible observability layer that records execution traces for every request. By capturing

inputs, outputs, and metadata at each intermediate step (including model calls, tool invocations, and control-flow decisions), it transforms the “black box” of an agent into a transparent process.

With tracing enabled, you’re no longer flying blind. Instead of receiving an incorrect final answer and guessing what went wrong, you can see the exact path the agent took to reach that response. This makes it much easier to debug failures, troubleshoot unexpected behavior, and assess performance as your application inevitably changes.

Let’s examine the specific components that make up an MLflow trace.

Anatomy of a Trace

At a high level, a trace is conceptually similar to an MLflow run (which we discussed in [Chapter 3](#)). Both belong to an MLflow experiment. The difference is that traditional experiments track model training cycles, whereas GenAI experiments use traces to capture live or simulated agent executions.

Each trace consists of two main parts: trace info and trace data (see [Figure 8-2](#)).

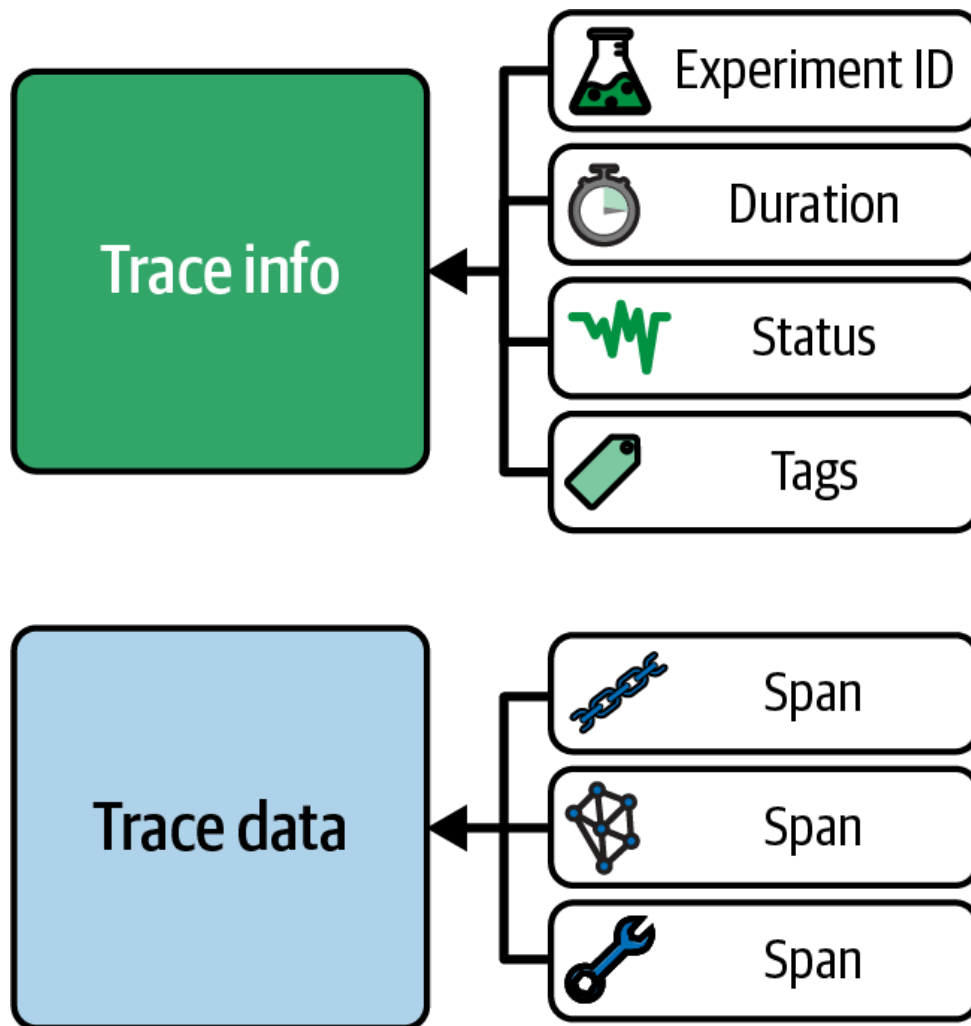


Figure 8-2. Trace info and trace data, adapted from [MLflow documentation](#)

Trace info contains high-level metadata about an execution. This includes the experiment ID, the trace status (for example, success or failure), and the total execution latency. It also supports *tags*, which add contextual information such as the user, session ID, and developer-defined key-value pairs. These tags are particularly useful for searching, filtering, and organizing traces during analysis, monitoring, and debugging.

Trace data stores the actual execution payload. It contains the instrumented *span objects* that capture the step-by-step behavior of your application from input to output. Spans record critical operations such as LLM invocations, tool calls, and intermediate processing steps, to provide a detailed, end-to-end view of how a GenAI application executes in practice.

MLflow's span object is designed to be fully compatible with the OpenTelemetry span specification. It is implemented as a dataclass, and it closely mirrors the OpenTelemetry span model, while adding convenience accessors and helper methods that are tailored specifically to GenAI workloads. Each span records essential attributes such as inputs and outputs, start and end times, the execution status, the span ID, the trace ID, and an optional parent ID for hierarchical traces.

MLflow supports a rich set of span types that record common operations within GenAI applications, including the following:

LLM

This records language model inference.

TOOL

This records the execution of external tools, functions, and APIs.

RETRIEVER

This records the retrieval of relevant context, for example, from an AI search index.

MEMORY

This records instances of reading from or writing to conversational memory.

EMBEDDING

This records the generation of vector embeddings from text or other inputs.

RERANKER

This records the reranking of retrieved results based on relevance.

As with traditional ML experiment tracking, MLflow can automatically log traces. However, autologging is not always sufficient if you want to attach custom metadata and domain-specific context. To gain full control, you must use the MLflow tracing APIs.

MLflow Tracing APIs

The simplest way to trace a function is by using the `@mlflow.trace()` decorator. When we apply it, it creates a span for the duration of the decorated function and automatically captures the function's inputs and outputs. For example, the following code produces a span named `my_func` that records input arguments `x` and `y`, along with the function's return value. Note that we first set an experiment to define the location of the traces and set the tracking URI in the same way we did in [Chapter 3](#):

```
import mlflow
from arxiv_curator.utils.common import set_mlflow_tracking_uri
```

```

set_mlflow_tracking_uri()
mlflow.set_experiment('/Shared/tracing-demo')

@mlflow.trace
def my_func(x, y):
    return x + y

```

We can do the same with the `mlflow.start_span()` function, which manually defines the span and its scope:

```

def my_func(x, y):
    return x + y

with mlflow.start_span("my_func") as span:
    x = 1
    y = 2
    span.set_inputs({"x": x, "y": y})
    result = my_function(x, y)
    span.set_outputs({"output": result})

```

We can also add more context to a trace, even when we're using the `@mlflow.trace()` decorator. For example, we may want to attach a session ID and request ID to the trace and later pass the same information to the agent endpoint so we can link the two. We can do this by using the `mlflow.update_current_trace()` function. The following example shows how to add a session ID, Git SHA, request ID, and tags related to the Model Serving endpoint:

```

import random
from datetime import datetime

timestamp = datetime.now().strftime("%Y%m%d-%H%M%S")
session_id = f"s-{timestamp}-{random.randint(100000, 999999)}"
request_id = f"req-{timestamp}-{random.randint(100000, 999999)}"

git_sha = "abcd"

@mlflow.trace
def my_func(x, y):
    mlflow.update_current_trace(
        metadata={
            "mlflow.trace.session": session_id,
        },
        tags={
            "model_serving_endpoint_name": "arxiv-agent-endpoint",
            "model_version": 1,
            "git_sha": git_sha,
        },
    )

```

```

        client_request_id=request_id
    )
    return x + y

```

We can search generated traces by using a workflow that's similar to MLflow experiment run searches. We can search traces by nearly any field. For example, the code below demonstrates how to search for traces using the `git_sha` tag:

```
mlflow.search_traces(filter_string=f"tags.git_sha = '{git_sha}'")
```

Now that we understand how to work with MLflow tracing and have already discussed MLflow flavors in the context of LLM applications, we can move on to defining a custom agent, applying the `@mlflow.trace` decorator, and passing additional context.

Defining a Custom Agent

The `arxivAgent` class inherits from the `mlflow.pyfunc.ResponsesAgent`. This base class requires us to implement prediction logic that produces `ResponsesAgentResponse` objects (or streams of response events).

Initialization and Async Handling

To initialize the class, we provide the Databricks Model Serving endpoint name, a system prompt, and the MCP tools we defined earlier. During this stage, we use the `WorkspaceClient` to retrieve an OpenAI-compatible client for Model Serving.

This setup ensures that the agent has everything it needs to orchestrate LLM calls and tool invocations. Notice that we set the MLflow experiment here and also use the `create_mcp_tools()` function we defined earlier. Here, we use the `nest_asyncio` package, which allows async code to run in environments where an event loop is already running. Python does not allow nested event loops, and calling `asyncio.run()` would raise an error. The `nest_asyncio.apply()` function patches the existing loop, enabling async functions like `create_mcp_tools()` to execute from synchronous code:

```

from mlflow.entities import SpanType
from mlflow.models.resources import (
    DatabricksGenieSpace,
    DatabricksServingEndpoint,
    DatabricksSQLWarehouse,
    DatabricksTable,
    DatabricksVectorSearchIndex,
)
from mlflow.pyfunc import ResponsesAgent
from mlflow.types.responses import (
    ResponsesAgentRequest,
    ResponsesAgentResponse,
    ResponsesAgentStreamEvent,
    output_to_responses_items_stream,
    to_chat_completions_input,
)

from arxiv_curator.config import ProjectConfig
from arxiv_curator.mcp import create_mcp_tools
from arxiv_curator.memory import LakebaseMemory

# Additional imports omitted for brevity - consult the book code

class ArxivAgent(ResponsesAgent):
    def __init__(
        self,
        llm_endpoint: str,
        system_prompt: str,
        catalog: str,
        schema: str,
        genie_space_id: str | None = None,
        lakebase_project_id: str | None = None,
    ):
        """Initializes the Arxiv Agent."""
        nest_asyncio.apply()

        self.system_prompt = system_prompt
        self.llm_endpoint = llm_endpoint
        self.workspace_client = WorkspaceClient()
        self.model_serving_client = (
            self.workspace_client.serving_endpoints.get_open_ai_client()
        )

        # Initialize Lakebase memory if configured
        self.memory: LakebaseMemory | None = None
        if lakebase_project_id:
            self.memory = LakebaseMemory(
                project_id=lakebase_project_id,
            )

```

```

# Create tools from config
host = self.workspace_client.config.host
tools = asyncio.run(create_mcp_tools(
    w=self.workspace_client,
    url_list=[
        f"{host}/api/2.0/mcp/vector-search/{catalog}/{schema}",
        f"{host}/api/2.0/mcp/genie/{genie_space_id}",
    ],
))
self._tools_dict = {tool.name: tool for tool in tools}

```

If a Lakebase project ID is provided, the `LakebaseMemory` class gets initialized so that the agent can retrieve and store messages using a session ID.

Core Agent Logic

The `get_tool_specs()` method returns the tool specifications in the format that's expected by OpenAI-style chat completions. Each `ToolInfo` object exposes a spec that describes the tool's name, parameters, and purpose. These specifications are then passed to the LLM so that it can decide when and how to call a tool during generation:

```

def get_tool_specs(self) -> list[dict]:
    """Returns tool specifications in the format OpenAI expects."""
    return [tool_info.spec for tool_info in self._tools_dict.values()]

```

This method executes a tool by name, with the arguments provided by the model. It looks up the corresponding `ToolInfo` object and calls its execution function. (While you may notice tracing annotations here, we'll intentionally skip explaining them for now and revisit them in the next section.) Conceptually, this method forms the bridge between the LLM's structured tool calls and your application logic. You'll also notice that we use the `@mlflow.trace` decorator to trace the tool execution:

```

@mlflow.trace(span_type=SpanType.TOOL)
def execute_tool(self, tool_name: str, args: dict) -> Any:
    """Executes the specified tool with the given arguments."""
    return self._tools_dict[tool_name].exec_fn(**args)

```

The `call_llm` method invokes the LLM endpoint and streams its output, yielding partial response chunks as they arrive. It uses exponential back-

off to retry requests in case of rate-limiting errors. The method converts internal messages into the chat completions format that's expected by the OpenAI-compatible API, attaches tool specifications, and enables streaming.

For observability, we manually create an MLflow span by using `mlflow.start_span()` instead of relying on `@mlflow.trace`. This allows us to capture additional information, such as the request ID from the response headers, and explicitly set span outputs after the stream completes. We've included the request ID so you can trace which MLflow span corresponds to a specific LLM call, which makes debugging and monitoring much easier:

```
@backoff.on_exception(backoff.expo, openai.RateLimitError)
def call_llm(
    self,
    messages: list[dict[str, Any]],
) -> Generator[dict[str, Any], None, None]:
    with warnings.catch_warnings():
        warnings.filterwarnings(
            "ignore", message="PydanticSerializationUnexpectedValue"
        )
    stream = self.model_serving_client.chat.completions.create(
        model=self.llm_endpoint,
        messages=to_chat_completions_input(messages),
        tools=self.get_tool_specs(),
        stream=True,
    )
    with mlflow.start_span(name="call_llm", span_type=SpanType.LLM) as span:
        span.set_inputs({"messages": messages})
        last_chunk: dict[str, Any] = {}
        for chunk in stream:
            chunk_dict = chunk.to_dict()
            last_chunk = chunk_dict
            yield chunk_dict
        llm_request_id = stream.response.headers.get("x-request-id")
        outputs: dict[str, Any] = {
            "model": last_chunk.get("model"),
            "usage": last_chunk.get("usage"),
        }
        if llm_request_id:
            outputs["llm_request_id"] = llm_request_id
        span.set_outputs(outputs)
```

When the model decides to call a tool, the `handle_tool_call` method handles the execution flow. The tool arguments are parsed from JSON, and the corresponding tool gets executed. The tool output is wrapped into

a function-call output item and gets appended to the running message history. The `ResponsesAgentStreamEvent` is then returned, to complete the tool call:

```
def handle_tool_call(
    self, tool_call: dict[str, Any], messages: list[dict[str, Any]]
) -> ResponsesAgentStreamEvent:
    """
    Execute tool calls, add them to the running message history,
    and return a ResponsesStreamEvent w/ tool output
    """
    args = json.loads(tool_call["arguments"])
    result = str(self.execute_tool(tool_name=tool_call["name"], args=args))

    tool_call_output = self.create_function_call_output_item(
        tool_call["call_id"], result
    )
    messages.append(tool_call_output)
    return ResponsesAgentStreamEvent(
        type="response.output_item.done", item=tool_call_output
    )
```

The `arxivAgent` class provides dedicated methods for loading and saving messages to memory. While this may seem redundant, it ensures that each operation is properly traced via the `mlflow.trace` decorator. By isolating memory access in these methods, both loading and saving are captured as separate trace spans, to improve observability and make debugging easier:

```
@mlflow.trace(span_type=SpanType.RETRIEVER, name="memory_load")
def load_memory(self, session_id: str) -> list[dict[str, Any]]:
    """Load previous messages from Lakebase memory."""
    if self.memory:
        return self.memory.load_messages(session_id)
    return []

@mlflow.trace(span_type=SpanType.CHAIN, name="memory_save")
def save_memory(
    self, session_id: str, messages: list[dict[str, Any]]
) -> None:
    """Save new messages to Lakebase memory."""
    self.memory.save_messages(session_id, messages)
```

The `call_and_run_tools()` method implements the core agent loop. It builds the message history from the system prompt, optional previous

messages (loaded from memory), and the current request, to enable stateful interactions.

It then runs `_run_tool_loop()`, which alternates between executing tool calls and invoking the LLM until a final response is produced or a maximum number of steps is reached. For simplicity, we omit the implementation of `_run_tool_loop()` here, but it's available in the full book code.

The method is traced with `@mlflow.trace` and enriched via `mlflow.update_current_trace()`, which adds tags such as `git_sha`, `model_version`, and `model_serving_endpoint_name`. These environment-based tags ensure that every run is traceable and make it clear whether the code was executed locally or behind a serving endpoint.

Finally, if memory is enabled, the method saves both the input and generated outputs, to allow future calls to continue the conversation seamlessly:

```
@mlflow.trace(span_type=SpanType.CHAIN)
def call_and_run_tools(
    self,
    request_input: list[dict[str, Any]],
    previous_messages: list[dict[str, Any]] | None = None,
    request_id: str | None = None,
    session_id: str | None = None,
) -> list[ResponsesAgentStreamEvent]:
    messages: list[dict[str, Any]] = [
        {"role": "system", "content": self.system_prompt},
    ]
    if previous_messages:
        messages.extend(previous_messages)
    messages.extend(request_input)

    mlflow.update_current_trace(
        tags={
            "git_sha": os.getenv("GIT_SHA", "local"),
            "model_serving_endpoint_name": os.getenv(
                "MODEL_SERVING_ENDPOINT_NAME", "local"),
            "model_version": os.getenv("MODEL_VERSION", "local"),
        },
        metadata=(
            {"mlflow.trace.session": session_id} if session_id else {}),
        client_request_id=request_id,
    )

    events = self._run_tool_loop(messages)
```

```

        if session_id and self.memory and request_input:
            self.save_memory(
                session_id,
                request_input + self._extract_output_items(events),
            )
        return events

```

Prediction and Streaming

The `predict()` method implements the nonstreaming interface required by `ResponsesAgent`. It simply calls `predict_stream()`, collects the emitted events, and returns the final outputs as a `ResponsesAgentResponse`. This makes it suitable for synchronous or batch use cases.

The `predict_stream()` method serves as the main entry point for execution. It extracts the `session_id` and `request_id` from the request, loads any previous messages from memory, and converts the incoming inputs into the internal message format.

The `predict_stream()` method delegates all core execution logic to `call_and_run_tools`, which we discussed earlier. It then yields the resulting events as they are produced:

```

def predict(self, request: ResponsesAgentRequest) -> ResponsesAgentResponse:
    events = list(self.predict_stream(request))
    return ResponsesAgentResponse(
        output=self._extract_output_items(events),
        custom_outputs=request.custom_inputs,
    )

@mlflow.trace(span_type=SpanType.AGENT)
def predict_stream(
    self, request: ResponsesAgentRequest
) -> Generator[ResponsesAgentStreamEvent, None, None]:
    custom = request.custom_inputs or {}
    session_id = custom.get("session_id")
    request_id = custom.get("request_id")

    previous_messages = (
        self.load_memory(session_id)
        if session_id and self.memory else []
    )

    request_input = [i.model_dump() for i in request.input]
    events = self.call_and_run_tools(

```

```

        request_input=request_input,
        previous_messages=previous_messages,
        request_id=request_id,
        session_id=session_id,
    )
    yield from events

```

The agent is defined as part of the package in the `arxiv_curator.agent` module. Let's now use this module and see how the agent works.

Testing the Agent

Let's take a look at the *chapter8_2.test_agent.py* notebook. Here, we start by importing the project configuration and initializing the agent. Key settings—such as the MLflow experiment name, the Lakebase project ID, which LLM endpoint to use, and the system prompt—are defined in the *project_config.yml* file:

```

import random
from datetime import datetime

import mlflow

from arxiv_curator.agent import ArxivAgent
from arxiv_curator.config import ProjectConfig
from arxiv_curator.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()

cfg = ProjectConfig.from_yaml("../project_config.yml")
mlflow.set_experiment(cfg.experiment_path)

agent = ArxivAgent(
    llm_endpoint=cfg.llm_endpoint,
    system_prompt=cfg.system_prompt,
    catalog=cfg.catalog,
    schema=cfg.schema,
    genie_space_id=cfg.genie_space_id,
    lakebase_project_id=cfg.lakebase_project_id
)

mlflow.models.set_model(agent)

```

We want to verify that the agent behaves as expected when custom inputs are provided, and that its memory is updated correctly. To do this, we call

the `predict()` method twice, using the same `session_id` for both requests.

This allows us to simulate a multiturn interaction, in which the second request can build on the context of the first. By reusing the same session, we can confirm that previous messages are successfully stored and retrieved from memory, to enable the agent to respond in a context-aware manner:

```
timestamp = datetime.now().strftime("%Y%m%d-%H%M%S")
session_id = f"s-{timestamp}-{random.randint(100000, 999999)}"

first_request = {
    "input": [
        {"role": "user",
         "content": "What are recent papers about LLMs and reasoning?"}
    ],
    "custom_inputs": {
        "session_id": session_id,
        "request_id": f"req-{timestamp}-1",
    },
}

result_1 = agent.predict(first_request)

second_request = {
    "input": [
        {"role": "user",
         "content": "Rewrite that post focusing on chain-of-thought prompting."}
    ],
    "custom_inputs": {
        "session_id": session_id,
        "request_id": f"req-{timestamp}-2",
    },
}

result_2 = agent.predict(second_request)
```

After we invoke the `predict` functions, we'll see that an MLflow trace has appeared under the defined experiment (see [Figure 8-3](#)):

Tags (3/3)								
Trace ID	Request	Response	Execution ...	Request ti...	State	git_sha	model_ser...	mod...
tr-0a5cfd...	Rewrite that post focusin...	🔥 **Chain-of-Thought Prompti...	4.221s	03/29/202...	🟢 OK	local	local	local
tr-52f7ca...	What are recent papers a...	🔥 **The Reasoning Renaissanc...	50.48s	03/29/202...	🟢 OK	local	local	local

Figure 8-3. Traces from the agent invocation

We can inspect the trace in the UI to view a detailed breakdown of the execution and the final output. By opening a trace, we can also confirm that the `session_id` we passed in the code is captured in the trace metadata (see [Figure 8-4](#)).

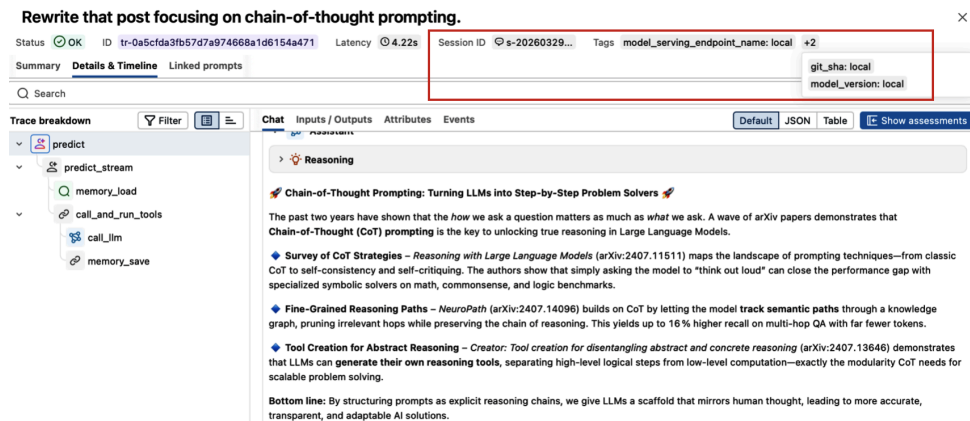


Figure 8-4. Inspecting the trace

LLM System Evaluation

Before we take a deep dive into the code, let's talk about what evaluating an LLM system means. Simply put, *evaluation* is the systematic process of measuring a system's quality. In traditional ML systems, we make this judgment by relying on well-established evaluation metrics, such as accuracy or the F1 score for classification and RMSE or R^2 for regression. LLM systems, however, are much harder to evaluate, which is why teams often treat *evals* (a term that practitioners commonly use to refer to LLM evaluations) as an afterthought.

During the LLM system development phase, teams can get pretty far with manual testing and going with their gut feelings. However, once the system reaches production and real users are using it, building without evals quickly becomes unsustainable. And after you receive complaints and make changes to the system, there will be no reliable way to know whether other behaviors have not silently regressed.

Just as we wouldn't skip evaluation in a standard ML workflow, we shouldn't skip it here. Evals are most valuable when implemented early, for several reasons:

Explicit behavior

They force you to define exactly what “good behavior” looks like for your agent.

Measurable impact

They allow you to measure how changes to orchestration logic, prompts, or the underlying model affect performance.

System optimization

In many ways, choosing a prompt or a temperature setting is like tuning hyperparameters in classic ML; evals provide the data you need to make those choices.

How to Evaluate LLM Systems

If you're starting with evals early, the best approach is a simple, iterative loop. Experts Aishwarya Naresh Reganti and Kiriti Badam, in their materials on AI Evals,¹ suggest this workflow:

1. Create examples: start small; 10–20 high-quality examples are better than 100 mediocre ones.
2. Run test cases: process them through your current agent.
3. Evaluate with the business: determine whether the output has met expectations.
4. Identify error patterns: explain exactly why an output failed.
5. Decide on key metrics: determine what you are actually measuring.
6. Iterate and expand: refine your prompts or tools and repeat.

Anthropic's [“Demystifying evals for AI agents” article](#) adds a few useful principles:

Product requirements naturally translate into test cases

If a requirement states that “*The agent must cite its sources,*” you can turn that into a test that verifies whether responses include references to the retrieved documents.

The “expert agreement” rule

A good test case is one in which two domain experts independently reach the same pass/fail verdict. If they disagree, the example is too noisy.

Test negative behaviors

Don't forget to test both cases in which a behavior should occur and where it shouldn't. Imbalanced datasets are just as problematic in agent evaluation as they are in traditional ML.

Table 8-1 illustrates how test cases for a technical support agent can be processed through an LLM system and evaluated alongside business stakeholders.

Table 8-1. Examples of test cases and outputs from an evaluation session

Input	System output	Expected response	Satisfactory	Reason
“Same issue as last month”	“Can you describe the problem?”	Reference previous tickets and confirm if there is a related issue.	No	Previous tickets were not retrieved, and the user was forced to repeat the context.
“Just a quick question about settings”	“Sure, which settings are you looking at? I can walk you through it.”	Respond proportionally and ask clarifying questions.	Yes	-
“Tried everything you suggested”	“Have you tried restarting?”	Acknowledge previous attempts and escalate or try a new approach.	No	An already-tried suggestion was repeated, and the conversational context was not maintained.

You can identify errors and potential causes after the initial evaluation session. For example, if the user says, “Same issue as last month,” and if the agent fails to retrieve the issue from the history, then the root cause may be missing access to ticket data. This isn’t a quick fix: it requires you to add a tool and integration, unlike with issues that you can address by changing the prompt alone.

As a next step, you should identify metrics. A *metric* represents a *behavior or quality you want to track over time*. Examples include tool usage correctness (for example, whether the agent attempted to retrieve data from history), tone appropriateness, number of tool calls, latency, and cost. For systems that produce structured output, common metrics include struc-

tured output correctness and simple quantitative measures such as output length and character count.

After you define metrics, you choose graders to measure them. A *grader* (also called a *scorer*) is the mechanism you use to evaluate a metric. In practice, agent evaluations typically rely on three types of graders:

Code-based graders

These are deterministic checks that are implemented directly in code. Examples include string and regular expression (regex) matching, character and token counts, output structure validation (for example, using Pydantic, as shown earlier), tool call verification, latency measurement, and cost tracking. Code-based graders are fast, inexpensive, objective, and reproducible, so they're ideal to introduce early and run continuously. Their main limitation is a lack of nuance: they are poorly suited to evaluating subjective or context-dependent behaviors.

Model-based graders

These are often referred to as *LLM judges*, and they use a language model to assess qualities that are difficult to capture with code alone, such as adherence to guidelines, correctness of reasoning, and appropriateness of text based on tone and context. These graders are more flexible and can handle open-ended outputs, but they are more expensive and nondeterministic. To make them reliable, you typically need to calibrate them against human judgment.

Human graders

Using these is unavoidable if you want to have an LLM system that truly meets business expectations. While slow and expensive, human evaluation, ideally performed by subject matter experts, serves as the gold standard. Human graders are essential for defining evaluation criteria, calibrating model-based graders, and reviewing edge cases. In production, they are often used for spot checks to ensure the system continues to behave as intended as user behavior and real-world conditions evolve.

It's also important to keep in mind that this is still a relatively young field. While terminology and naming conventions may vary across tools and platforms, the underlying principles remain consistent. Each framework introduces its own vocabulary for similar concepts.

With that context in mind, let's take a closer look at the MLflow GenAI evaluation framework.

MLflow GenAI Evaluation Framework

Scorers serve as the core components of the MLflow GenAI evaluation framework. To help you get started, MLflow provides several *predefined model-based scorers* that are designed to assess different aspects of model or agent behavior.

At a minimum, every evaluation requires scorers to have access to two things: *inputs* (what was provided to the system) and *outputs* (what the system produced). These correspond to the first and second columns of our earlier test cases in [Table 8-1](#). In addition, you must also specify the *evaluation model* that will act as the judge. For some scorers, this information is sufficient: metrics like *RelevanceToQuery*, *Completeness*, *Fluency*, and *Safety* can evaluate a response using only the inputs and outputs, without requiring any additional annotations.

Other scorers rely on *user-provided evaluation criteria*. For example, *Correctness* and *Equivalence* require explicit annotations, such as reference answers, expected facts, or task-specific expectations. This is similar to the information shown in column 3 of [Table 8-1](#).

Finally, some scorers operate on agent behavior rather than just text output. *Tool-calling* and *retrieval-related scorers* need access to traces so they can inspect tool usage, parameters, and execution order. In addition, certain scorers that measure conversation guidelines or role adherence are *session based*, meaning they evaluate the entire conversation or interaction sequence instead of individual input-output pairs.

[Table 8-2](#) summarizes MLflow's predefined model-based scorers.

Table 8-2. MLflow scorers^a

Scorer	What it evaluates	Does it require expectations traces a session?
RelevanceToQuery	Does the response directly address the user's input?	No No No
Correctness	Are the expected facts supported by the response?	Yes No No
Completeness	Does the agent address all questions in a single user prompt?	No No No
Fluency	Is the response grammatically correct and naturally flowing?	No No No
ExpectationsGuidelines	Does the response meet specific expectations and guidelines?	Yes No No
Safety	Is the response free of harmful or toxic content?	No No No
Equivalence	Is the response equivalent to the expected output?	Yes No No
RetrievalGroundedness	Is the response grounded in retrieved information?	No Yes No
RetrievalRelevance	Are retrieved documents relevant to the user's request?	No Yes No
RetrievalSufficiency	Do retrieved documents contain all necessary information?	Yes Yes No

Scorer	What it evaluates	Does it require expectations traces a session?
ToolCallCorrectness	Are the tool calls and arguments correct for the user query?	No Yes No
ToolCallEfficiency	Are the tool calls efficient without redundancy?	No Yes No
ConversationCompleteness	Does the agent address all user questions throughout the conversation?	No Yes Yes
ConversationalGuidelines	Do the assistant's responses comply with provided guidelines?	No Yes Yes
ConversationalRoleAdherence	Does the assistant maintain its assigned role throughout the conversation?	No Yes Yes
ConversationalSafety	Are the assistant's responses safe and free of harmful content?	No Yes Yes
ConversationalToolCallEfficiency	Was tool usage across the conversation efficient and appropriate?	No Yes Yes
KnowledgeRetention	Does the assistant correctly retain information from earlier user inputs?	No Yes Yes
UserFrustration	Is the user frustrated and was any frustration resolved?	No Yes Yes

Scorer	What it evaluates	Does it require expectations traces a session?
--------	-------------------	--

^a Adapted from [MLflow documentation](#) where you can find an overview of all available scorers.

Predefined scorers in MLflow evaluate model behavior by using instructions that are built directly into the library. They are useful for getting started quickly and for generating early evaluation signals, but on their own, they are rarely sufficient for rigorous or production-grade evaluation.

To support more tailored evaluation needs, MLflow allows you to define *custom scorers*. The simplest of these is the *Guidelines scorer*, which lets you specify general, natural language rules that apply to all tasks without requiring per-example expectations. Because guidelines are evaluated as pass/fail conditions, this scorer provides a fast and accessible way to customize evaluation behavior.

For more nuanced and flexible evaluation, MLflow offers *template-based LLM scorers*. These scorers enable richer evaluation logic by allowing the judge to reason over additional context, such as expectations, conversation history, and execution traces.

[Figure 8-5](#) compares the different types of LLM-based scorers and their complexity levels.

Simplicity versus control and use cases

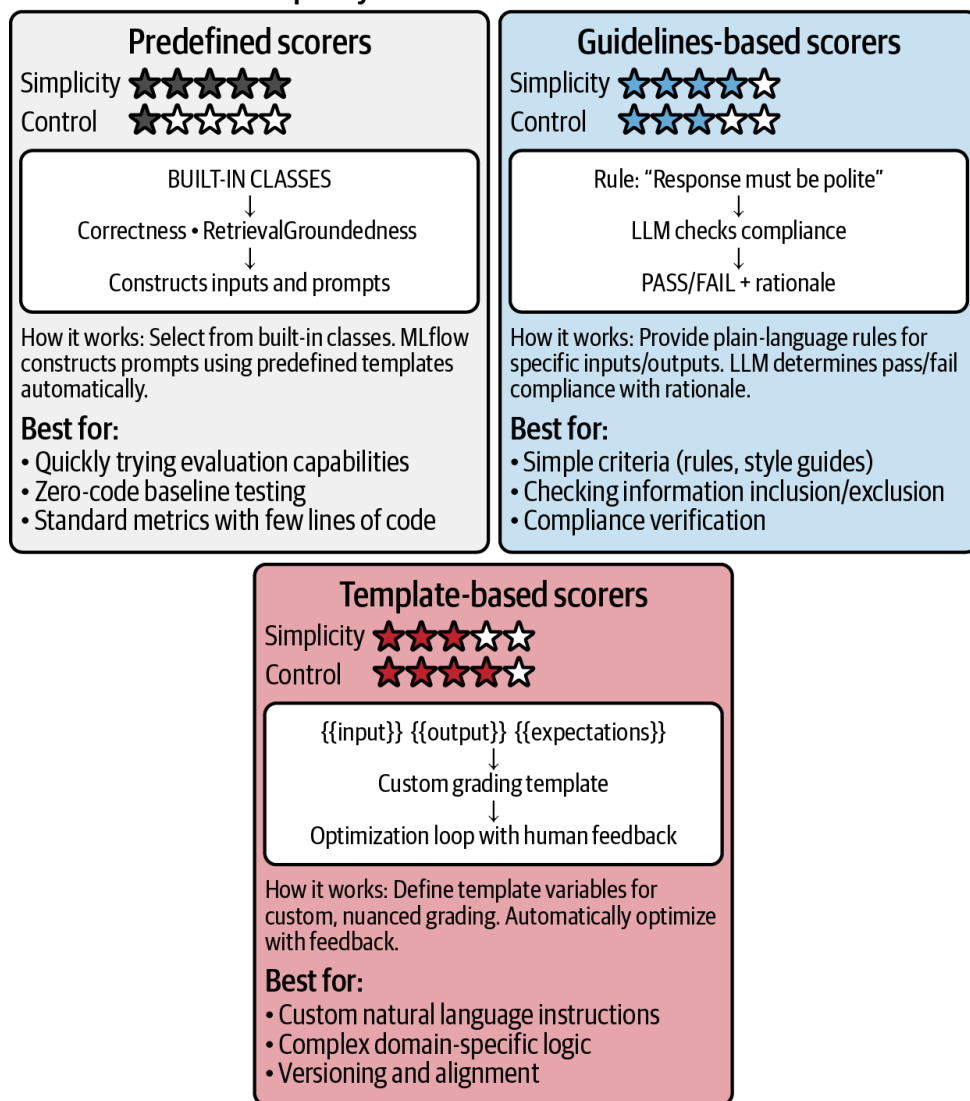


Figure 8-5. MLflow LLM scorer approaches

The differences between guidelines-based and template-based LLM scorers are not always obvious at first glance. To make these distinctions concrete, we'll compare the Guidelines scorer with a template-based LLM scorer we'll create by using the `make_judge` function. We'll start with a simple example:

```
import mlflow
from mlflow.genai.scorers import Guidelines

mlflow.set_experiment("/Shared/guidelines-example")
escalation_guidelines = Guidelines(
    name="escalation_handling",
    guidelines=[
        "When a user indicates previous attempts failed, the response must "
        "acknowledge their efforts and either escalate or offer a new approach"
    ],
    model="databricks:/databricks-gpt-oss-120b"
)
```

```

data = [
    {
        "inputs": {"message": "Tried everything you suggested"},
        "outputs": "Have you tried restarting?",
    },
    {
        "inputs": {"message": "Tried everything you suggested"},
        "outputs": (
            "I understand you've already tried the previous suggestions "
            "without success. Let me escalate this to our senior support "
            "team who can look into this more deeply. In the meantime, "
            "could you share any error logs you've encountered?"
        ),
    },
]

mlflow.genai.evaluate(data=data, scorers=[escalation_guidelines])

```

This evaluation session creates traces that show the first example failed the guideline and the second passed it (see [Figure 8-6](#)).

						Assessments (1)
Trace ID	Request	Response	Execution t...	Request ti...	State	escalation_ha...
						PASS 50%
tr-4fec6e...	{"message": "Tried everything you suggest...	"I understand you've already tried the previ...	0s	02/01/2026...	OK	Pass
tr-b1a1db...	{"message": "Tried everything you suggest...	"Have you tried restarting?"	0s	02/01/2026...	OK	Fail

Figure 8-6. Traces: escalation handling based on guidelines

The Guidelines scorer produces binary outcomes (pass/fail). When you need more expressive feedback, such as a 1–5 rating, a continuous score, or structured diagnostic output, you should use a template-based LLM scorer. While this is just one example of when an LLM scorer is the better choice, it represents a common and practical scenario.

In the following code snippet, we create a judge that produces an escalation quality score (from 1 to 5) instead of a binary output. Notice that the `make_judge()` function takes an instruction argument that contains template variables. Template-based LLM scorers support `{{ inputs }}`, `{{ outputs }}`, and `{{ expectations }}` template variables:

```

import mlflow
from mlflow.genai.judges import make_judge

# Example using make_judge for escalation quality evaluation (1-5 scale)
mlflow.set_experiment("/Shared/make-judge-example")
escalation_judge = make_judge(
    name="escalation_quality",

```

```

instructions=(
    "Evaluate how well the response in {{ outputs }} handles the user's "
    "message in {{ inputs }} when they indicate previous attempts failed. "
    "Score from 1 to 5:\n"
    "1 - Ignores context, repeats already-tried suggestions\n"
    "2 - Acknowledges frustration but offers no new solutions\n"
    "3 - Offers a new approach but lacks empathy or clarity\n"
    "4 - Acknowledges efforts and offers a reasonable new approach\n"
    "5 - Empathetic, escalates appropriately or provides creative solution"
),
model="databricks:/databricks-gpt-oss-120b",
feedback_value_type=int,
)

mlflow.genai.evaluate(data=data, scorers=[escalation_judge])

```

This evaluation session creates traces that show scores of 5 and 1 (respectively) for the two examples we provided (see [Figure 8-7](#)).

						Assessments (1)
Trace ID	Request	Response	Execution t...	Request ti...	State	escalation_qu...
						STRING 2 values 1 50% 5 50%
tr-2404e1...	{ "message": "Tried everything you suggest...	"I understand you've already tried the previ...	0s	02/01/2026...	🟢 OK	5
tr-448bf4...	{ "message": "Tried everything you suggest...	"Have you tried restarting?"	0s	02/01/2026...	🟢 OK	1

Figure 8-7. Traces: escalation quality based on a template-based LLM scorer

One of the key benefits of using a template-based LLM scorer is that it lets you better align automated evaluation with human judgment. You can add human evaluations directly through the MLflow UI and then combine those annotations with trace data and LLM-based judge evaluations to align the scorer.

In this example, we use the Stochastic Introspective Mini-Batch Ascent ([SIMBA](#)) optimizer from the DSPy library. It enables an LLM to analyze its own performance and derive improvement rules by sampling mini-batches, identifying challenging examples with high output variability, and either generating self-reflective rules or incorporating successful examples as demonstrations. Custom optimizers are also supported.

To effectively align the judge, it is recommended to have at least 10 traces available for optimization:²

```

experiment_id = mlflow.search_experiments(
    filter_string="name='/Shared/make-judge-example'")[0].experiment_id

```

```

from mlflow.genai.judges.optimizers import SIMBAAAlignmentOptimizer

# Retrieve traces with both judge and human assessments
traces_for_alignment = mlflow.search_traces(
    experiment_ids=[experiment_id],
    return_type="list"
)

# Filter for traces with both judge and human feedback
valid_traces = []
for trace in traces_for_alignment:
    feedbacks = trace.search_assessments(name="escalation_quality")
    has_judge = any(f.source.source_type == "LLM_JUDGE" for f in feedbacks)
    has_human = any(f.source.source_type == "HUMAN" for f in feedbacks)
    if has_judge and has_human:
        valid_traces.append(trace)

optimizer = SIMBAAAlignmentOptimizer(
    model="databricks:/databricks-gpt-oss-120b"
)

aligned_judge = escalation_judge.align(optimizer, valid_traces)

```

MLflow also supports *Agent-as-a-Judge*, for which we can use the same `make_judge()` function and pass an instruction with the `{{ trace }}` template variable. Here is an example of an Agent-as-a-Judge that uses trace to evaluate performance of the system:

```

from typing import Literal

performance_judge = make_judge(
    name="performance_analyzer",
    instructions=(
        "Analyze the {{ trace }} for performance issues.\n\n"
        "Check for:\n"
        "- Operations taking longer than 2 seconds\n"
        "- Redundant API calls or database queries\n"
        "- Inefficient data processing patterns\n"
        "- Proper use of caching mechanisms\n\n"
        "Rate as: 'optimal', 'acceptable', or 'needs_improvement'"
    ),
    feedback_value_type=Literal["optimal", "acceptable", "needs_improvement"],
    model="databricks:/databricks-gpt-oss-120b",
)

```

The key difference between an LLM-as-a-Judge and an Agent-as-a-Judge is that the latter has access to tools for inspecting traces stored in the MLflow backend. This allows it to retrieve high-level trace metadata, in-

spect individual spans, and search for patterns across spans using regular expressions.

In addition to complex evaluation mechanisms, MLflow supports *code-based scorers*, which allow you to implement custom evaluation logic directly in Python. These are best suited for deterministic checks such as structural validation, tool usage verification, latency or cost thresholds, and other objective criteria that are difficult or unnecessary to express in natural language.

Beyond evaluation, scorers also provide the objective metrics needed for prompt optimization.³ MLflow supports several optimization techniques, including Multiprompt Instruction PProposal Optimizer Version 2 ([MIPROv2](#)) from the DSPy library.

MIPROv2 jointly refines task instructions and few-shot examples by bootstrapping candidate examples, proposing alternative instructions that reflect different task strategies, and selecting the best combination using Bayesian optimization. The optimizer can be used to tune both instructions and few-shot examples together or to optimize instructions alone in a zero-shot setting.

Prompt optimization is conceptually similar to hyperparameter tuning in traditional machine learning: it is an iterative process that you should revisit periodically as your system, prompts, and data evolve.

Evaluating, Logging, and Registering the Agent

To see evaluation in practice, we'll create a code-based scorer and a custom guideline. We are intentionally keeping these examples straightforward; the goal here isn't to build the "perfect" agent but to demonstrate the specific tools you need to operate an agent effectively. Just as we did with traditional models in [Chapter 3](#), we'll log these evaluation results directly to your MLflow experiment for easy tracking and comparison.

Finally, after we evaluate the agent, we can log it and make it ready for serving by registering it in Unity Catalog.

Evaluation in Practice

Before we can evaluate an agent, we need to provide evaluation examples. They can range from simple inputs to more detailed pairs that include both inputs and expectations. In our case, we'll go for a simple approach by providing only the inputs. Our test cases are stored in an *eval_inputs.txt* file that contains the following records:

```
What is so special about the last paper on RAG?
Summarize the key findings from the last paper on reasoning.
What problem does the last paper on multi-agent systems solve?
Tell me about the last paper on prompt engineering.
What are the main contributions of the last paper on LLM fine-tuning?
Explain the methodology in the last paper on chain-of-thought prompting.
What results did the last paper on hallucination detection achieve?
Describe the last paper on LLM benchmarks.
What approach does the last paper on code generation propose?
What conclusions does the last paper on embedding models draw?
```

In the *chapter8_4.evaluate_log_register_agent.py* notebook, we initialize the agent by following the same process as before, define the predict function used for evaluation, and load these records into memory:

```
import mlflow
from mlflow.genai.scorers import Guidelines

from arxiv_curator.agent import ArxivAgent
from arxiv_curator.config import ProjectConfig
from arxiv_curator.evaluation import (
    hook_in_post_guideline,
    polite_tone_guideline,
    word_count_check,
)

from arxiv_curator.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()

cfg = ProjectConfig.from_yaml("../project_config.yml")
mlflow.set_experiment(cfg.experiment_path)

agent = ArxivAgent(
    llm_endpoint=cfg.llm_endpoint,
    system_prompt=cfg.system_prompt,
    catalog=cfg.catalog,
    schema=cfg.schema,
    genie_space_id=cfg.genie_space_id,
```

```

        lakebase_project_id=cfg.lakebase_project_id,
    )

# Load evaluation inputs
with open("../eval_inputs.txt") as f:
    eval_data = [{"inputs": {
        "question": line.strip()}} for line in f if line.strip()]

def predict_fn(question: str) -> str:
    """Predict function that wraps the agent for evaluation."""
    request = {"input": [{"role": "user", "content": question}]}
    result = agent.predict(request)
    return result.output[-1].content

```

Scorers are defined under the evaluation module of our package. Here, we specify guidelines (we want the LinkedIn post to have an engaging hook and a polite, professional tone). We also define a code-based scorer that evaluates that the length of the post must be no longer than 350 words:

```

import mlflow
from mlflow.genai.scorers import Guidelines

polite_tone_guideline = Guidelines(
    name="polite_tone",
    guidelines=[
        "The response must use a polite and professional tone throughout",
        "The response should be friendly and helpful without being condescending",
        "The response must avoid any dismissive or rude language"
    ],
    model="databricks:/databricks-gpt-oss-120b"
)

hook_in_post_guideline = Guidelines(
    name="hook_in_post",
    guidelines=[
        "The response must start with an engaging hook that captures attention",
        "The opening should make the reader want to continue reading",
        "The response should have a compelling introduction before diving into de
    ],
    model="databricks:/databricks-gpt-oss-120b"
)

# Define code-based scorer for word count
@mlflow.genai.scorer
def word_count_check(outputs: list) -> bool:
    """Check that the output is under 350 words."""

```

```
word_count = len(outputs[0]["text"].split())
return word_count < 350
```

With the scorers and evaluation data in place, we can now run the evaluation:

```
mlflow.set_experiment("/Shared/genai-arxiv-agent")

results = mlflow.genai.evaluate(
    predict_fn=predict_fn,
    data=eval_data,
    scorers=[word_count_check,
             polite_tone_guideline,
             hook_in_post_guideline]
)
```

Once the evaluation is complete, you can retrieve the results directly from the `EvaluationResults` object, where they are stored within the `metrics` attribute. For a more visual deep dive, you can inspect the performance data in the MLflow tracing UI (see [Figure 8-8](#)). This view is particularly helpful as it provides a clear breakdown of how frequently the agent’s responses passed your defined assessments.

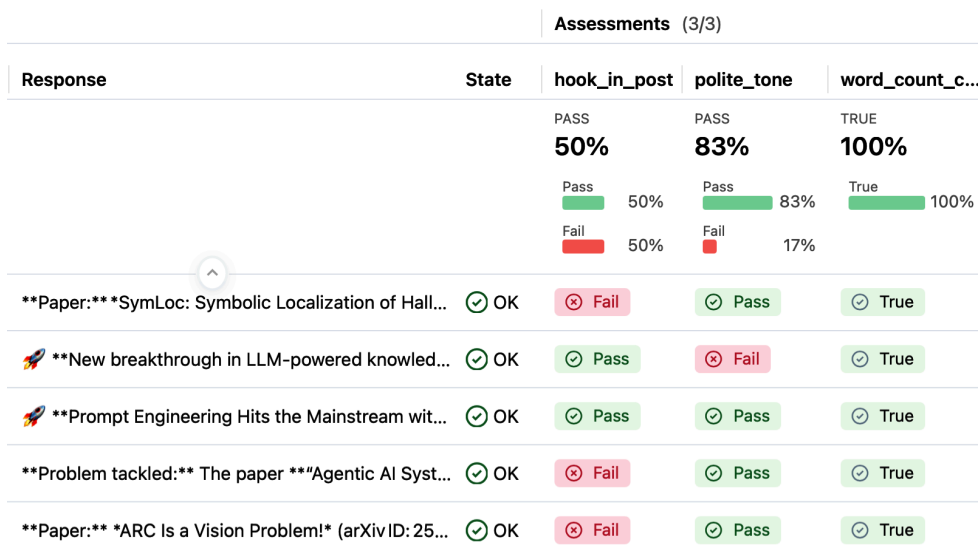


Figure 8-8. Evaluation results on the MLflow tracing UI

We can use the logic we defined to run evaluation before logging the model, as well as to monitor the system once it is in production. However, before we can explore those monitoring workflows in detail, we need to log and register the agent.

Agent Logging

Custom agents that inherit from the `mlflow.pyfunc.ResponsesAgent` class can be logged as code.⁴ This means that we must specify the agent, along with all its resources, in a Python file. In our project, the `arxiv_agent.py` file is located in the root of the repository. While the code is very similar to what we used for agent evaluation, there are a few key differences.

For instance, we specify `ModelConfig`, which is used for development purposes. You may notice that we are repeating the values from the `project_config.yml` file (except that we've replaced the prompt). Think of these values as placeholders; the actual values from the `project_config.yml` file will be passed in the model logging step. We also must use the `set_model()` function from the `mlflow.models` module, which allows MLflow to package the agent as code. Here is the structure of the `arxiv_agent.py` script:

```
import mlflow
from mlflow.models import ModelConfig

from arxiv_curator.agent import ArxivAgent

config = ModelConfig(
    development_config={
        "catalog": "mlops_dev",
        "schema": "arxiv",
        "genie_space_id": "01f0e97a42981382b3d16f3f1899fdb5",
        "system_prompt": "prompt placeholder",
        "llm_endpoint": "databricks-gpt-oss-120b",
        "lakebase_project_id": "arxiv-agent-lakebase",
    }
)

agent = ArxivAgent(
    llm_endpoint=config.get("llm_endpoint"),
    system_prompt=config.get("system_prompt"),
    catalog=config.get("catalog"),
    schema=config.get("schema"),
    genie_space_id=config.get("genie_space_id"),
    lakebase_project_id=config.get("lakebase_project_id"),
)

mlflow.models.set_model(agent)
```

Once you've defined the agent as code, we can proceed with model logging. Before doing so, however, you must define the model's *resource de-*

dependencies and configuration. Make sure to define all underlying dependencies, such as the tables Genie needs to access:

```
from datetime import datetime

import mlflow
from mlflow.models.resources import (
    DatabricksGenieSpace,
    DatabricksServingEndpoint,
    DatabricksVectorSearchIndex,
    DatabricksTable,
    DatabricksSQLWarehouse
)

from arxiv_curator.config import ProjectConfig

cfg = ProjectConfig.from_yaml("../project_config.yml")

model_name = f"{cfg.catalog}.{cfg.schema}.arxiv_agent"

resources = [
    DatabricksServingEndpoint(endpoint_name=cfg.llm_endpoint),
    DatabricksGenieSpace(genie_space_id=cfg.genie_space_id),
    DatabricksVectorSearchIndex(
        index_name=f"{cfg.catalog}.{cfg.schema}.arxiv_index"),
    DatabricksTable(table_name=f"{cfg.catalog}.{cfg.schema}.arxiv_papers"),
    DatabricksSQLWarehouse(warehouse_id=cfg.warehouse_id),
    DatabricksServingEndpoint(endpoint_name="databricks-bge-large-en"),
]

model_config = {
    "catalog": cfg.catalog,
    "schema": cfg.schema,
    "genie_space_id": cfg.genie_space_id,
    "system_prompt": cfg.system_prompt,
    "llm_endpoint": cfg.llm_endpoint,
    "lakebase_project_id": cfg.lakebase_project_id,
}
```

We can log the agent by using the same approach we used for the custom model we introduced in [Chapter 3](#). In this example, we start by providing placeholder values for `git_sha` and the Databricks `run_id` job, along with the path to the Python model relative to the notebook.

When the job executes, the script receives the actual values for these parameters at runtime, following the same configuration pattern we used in [Chapter 5](#):

```

git_sha = "abc"
run_id = "unset"

mlflow.set_experiment("/Shared/genai-arxiv-agent")
ts = datetime.now().strftime('%Y-%m-%d')
with mlflow.start_run(
    run_name=f"arxiv-mcp-agent-{ts}",
    tags={"git_sha": git_sha, "run_id": run_id}
) as run:
    model_info = mlflow.pyfunc.log_model(
        name="agent",
        python_model="../arxiv_agent.py",
        resources=resources,
        input_example=test_request,
        model_config=model_config
    )

```

Registering the Agent and Preparing for Serving

Once the model is logged, we can proceed with registering it:

```

model_name = f"{cfg.catalog}.{cfg.schema}.arxiv_agent"

registered_model = mlflow.register_model(
    model_uri=model_info.model_uri,
    name=model_name,
    tags={"git_sha": git_sha, "run_id": run_id},
    env_pack="databricks_model_serving"
)

```

Notice that we pass the `env_pack` argument with the `databricks_model_serving` value. This indicates that we will use the express deployment option (previously, serverless optimized deployment),⁵ which significantly simplifies the handling of custom dependencies.

Although the agent defined in the `arxiv_agent.py` script imports the custom `arxiv_curator` package, we do not log this package explicitly, unlike with the approach we used in [Chapter 2](#). The *serverless environment packaging* mechanism handles dependency resolution automatically. For this to work correctly, both the model logging and registration steps must run in the *same serverless environment*.

Simplifying custom packaging is not the primary benefit of this approach. The main advantage is significantly *faster agent deployment and autoscal-*

ing. Keep in mind that model registration takes slightly longer than usual in this case. Another important constraint to keep in mind is *model size*: Databricks serverless optimized deployments support a maximum model size of 1 GB, compared to 4 GB when environment packaging is not used.⁶

The environment packaging stores all Python dependencies together with the model. We can find them under the `_databricks` folder in the agent Artifacts tab (see [Figure 8-9](#)).

 **agent**

Overview

Traces

Artifacts

Logged model artifacts

▼  agent

>  __pycache__

▼  _databricks

 model_environment.tar

 model_version.tar

>  metadata

 MLmodel

 arxiv_agent.py

 conda.yaml

 input_example.json

 python_env.yaml

 registered_model_meta

 requirements.txt

 serving_input_example.json

Figure 8-9. `_databricks` folder with project dependencies

After we register the agent, we can set the latest-model alias so that we can easily retrieve the latest version of the agent:

```
from mlflow import MlflowClient

client = MlflowClient()
client.set_registered_model_alias(
    name=model_name,
    alias="latest-model",
    version=registered_model.version,
)
```

Now that we've successfully logged and registered the agent, let's talk about deployment.

Conclusion

In this chapter, we took the agent from a definition to a managed, observable model. We explored the MLflow flavors used to represent an agent, and then, we logged and registered it as a versioned model in Databricks Unity Catalog. We evaluated it with MLflow's GenAI evaluation tools to measure its quality and compare versions. And we added MLflow tracing, which records each step the agent takes: every retrieval, tool call, and model response.

A logged, evaluated, and traced agent is ready to leave the development environment. The next chapter covers deployment: serving the agent behind an endpoint and monitoring it in production, so the evaluation and tracing we set up here keep working once real traffic arrives.

- ¹ Check out [their repository](#).
- ² For more details, check out [the Databricks documentation on judge alignment](#).
- ³ Consult [the MLflow documentation](#) for more details on prompt optimization.
- ⁴ You can consult [the MLflow documentation on models from code](#).
- ⁵ For Model Serving endpoints, consult [the documentation on express deployments](#).
- ⁶ CPU workload sizes of 8 GB and 16 GB are also available with regular deployments, [per the documentation](#).

Previous chapter

< [7. Foundation Models and Context Engineering](#)

Next chapter

[9. Deploying in Monitoring LLM-Based Systems](#) >